

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия,
Системное и прикладное программное обеспечение

Дисциплина: Вычислительная математика

Лабораторная работа №2

Численное решение нелинейных уравнений и систем
Вариант №5

Выполнил:
Карнажицкий Максим Романович
Группа: Р3211

Проверил:
Малышева Т. А.

г. Санкт-Петербург, 2026

Содержание

1. Цель работы	2
2. Вычислительная реализация задачи	3
2.1. Решение нелинейного уравнения	3
2.1.1. Уточнение крайнего правого корня методом хорд	4
2.1.2. Уточнение крайнего левого корня методом простой итерации	4
2.1.3. Уточнение центрального корня методом секущих	6
2.2. Решение системы нелинейных уравнений	7
3. Программная реализация задачи	11
3.1. Функции решения нелинейных уравнений	11
3.1.1. Метод дихотомии	11
3.1.2. Метод Ньютона	12
3.1.3. Метод простых итераций	14
3.2. Решение системы нелинейных уравнений методом простых итераций	16
3.3. Результаты работы программы	19
4. Вывод	22

1. Цель работы

Изучить численные методы решения нелинейных уравнений и их систем, найти корни заданного нелинейного уравнения/системы нелинейных уравнений, выполнить программную реализацию методов.

2. Вычислительная реализация задачи

2.1. Решение нелинейного уравнения

Уравнение согласно варианту 5:

$$f(x) = -2,7x^3 - 1,48x^2 + 19,23x + 6,35$$

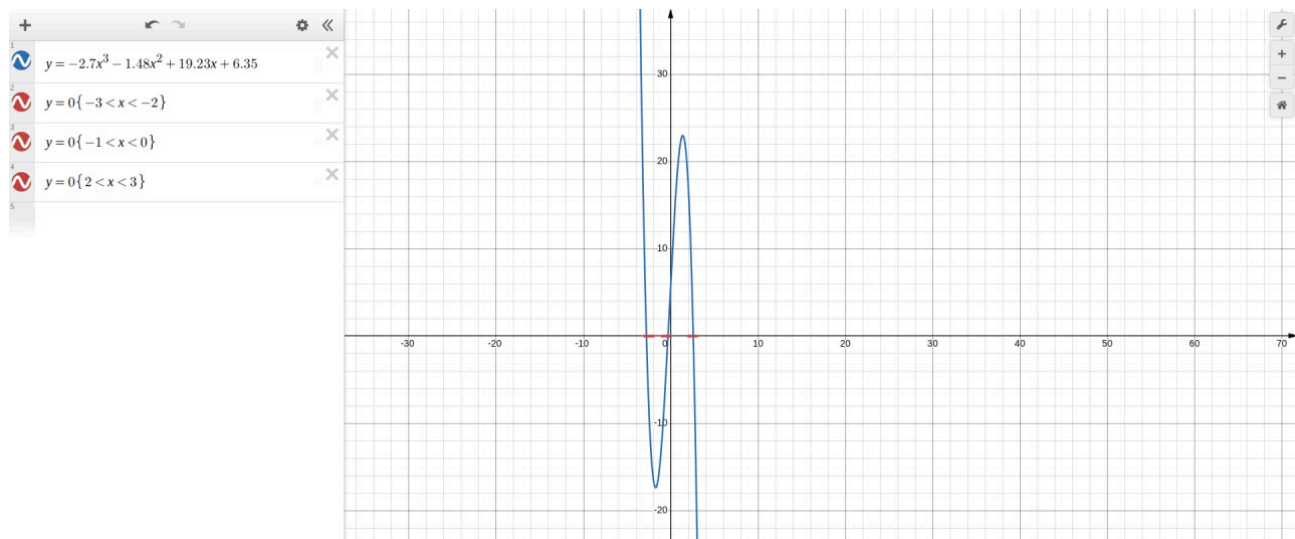


Рис. 1. График исходного уравнения

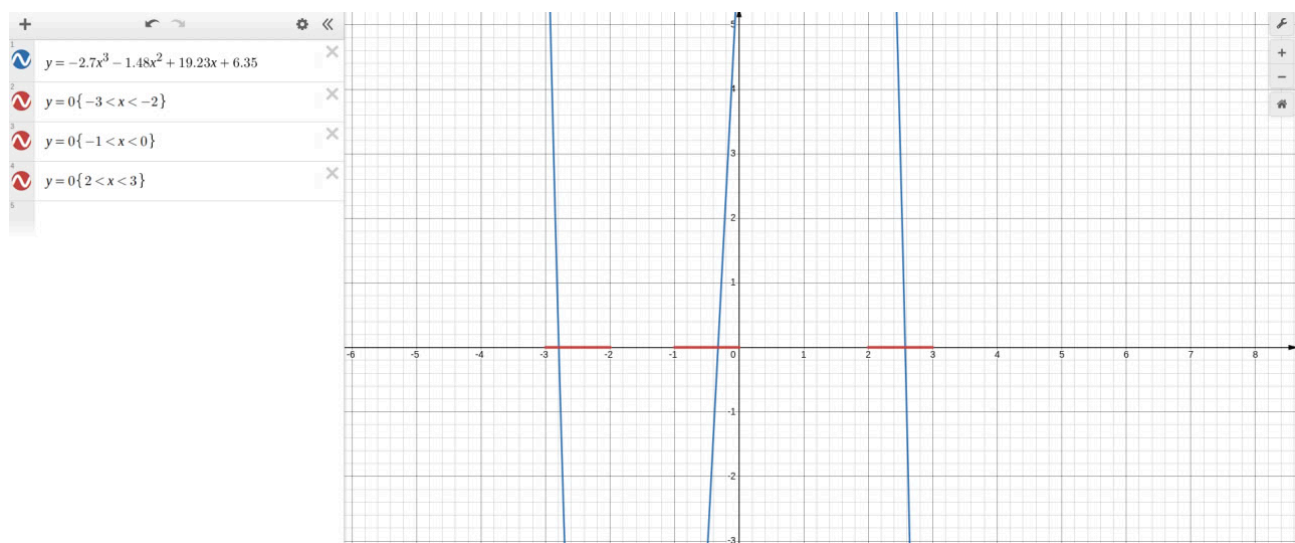


Рис. 2. График исходного уравнения в увеличенном масштабе

Интервалы изоляции корней (на рис. 2) выделены красными линиями на оси абсцисс):

1. $x \in [-3; -2]$
2. $x \in [-1; 0]$
3. $x \in [2; 3]$

Уравнение имеет 3 корня. Уточним каждый корень исходного уравнения с помощью следующих способов (согласно варианту):

- 1 корень: Метод хорд
 2 корень: Метод простой итерации
 3 корень: Метод секущих

$$\varepsilon = 10^{-2}$$

2.1.1. Уточнение крайнего правого корня методом хорд

Пример вычислений для первого шага:

$$a = 2, b = 3; f(a) = 17,29, f(b) = -22,18$$

Уравнение хорды, проходящей через точки $A(a, f(a)), B(b, f(b))$:

$$\frac{y - f(a)}{f(b) - f(a)} = \frac{x - a}{b - a}$$

Отсюда, точка пересечения хорды с осью абсцисс ($x = 0$):

$$x = a - \frac{b - a}{f(b) - f(a)} \cdot f(a)$$

$$x = 2 - \frac{3 - 2}{-22,18 - 17,29} \cdot 17,29 \approx 2,438$$

$$f(x) = f(2,438) \approx 5,308$$

№ шага	a	b	x	$f(a)$	$f(b)$	$f(x)$	$ x_{k+1} - x_k $
1	2	3	2,438	17,29	-22,18	5,310	—
2	2,438	3	2,547	5,310	-22,18	1,116	0.109
3	2,547	3	2,569	1,116	-22,18	0,206	0,022
4	2,569	3	2,573	0.206	-22,18	0,040	0.004

Таблица 1. Таблица вычислений для метода хорд

Можно заметить, что правая граница у нас была фиксирована, потому что было выполнено условие $f(b) \cdot f''(b) > 0$ – несложно проверить.

Ответ: 2,573 при $\varepsilon = 0,01$

2.1.2. Уточнение крайнего левого корня методом простой итерации

$$a = -3, b = -2$$

$$f(x) = -2,7x^3 - 1,48x^2 + 19,23x + 6,35$$

$$f'(x) = -8,1x^2 - 2,96x + 19,23$$

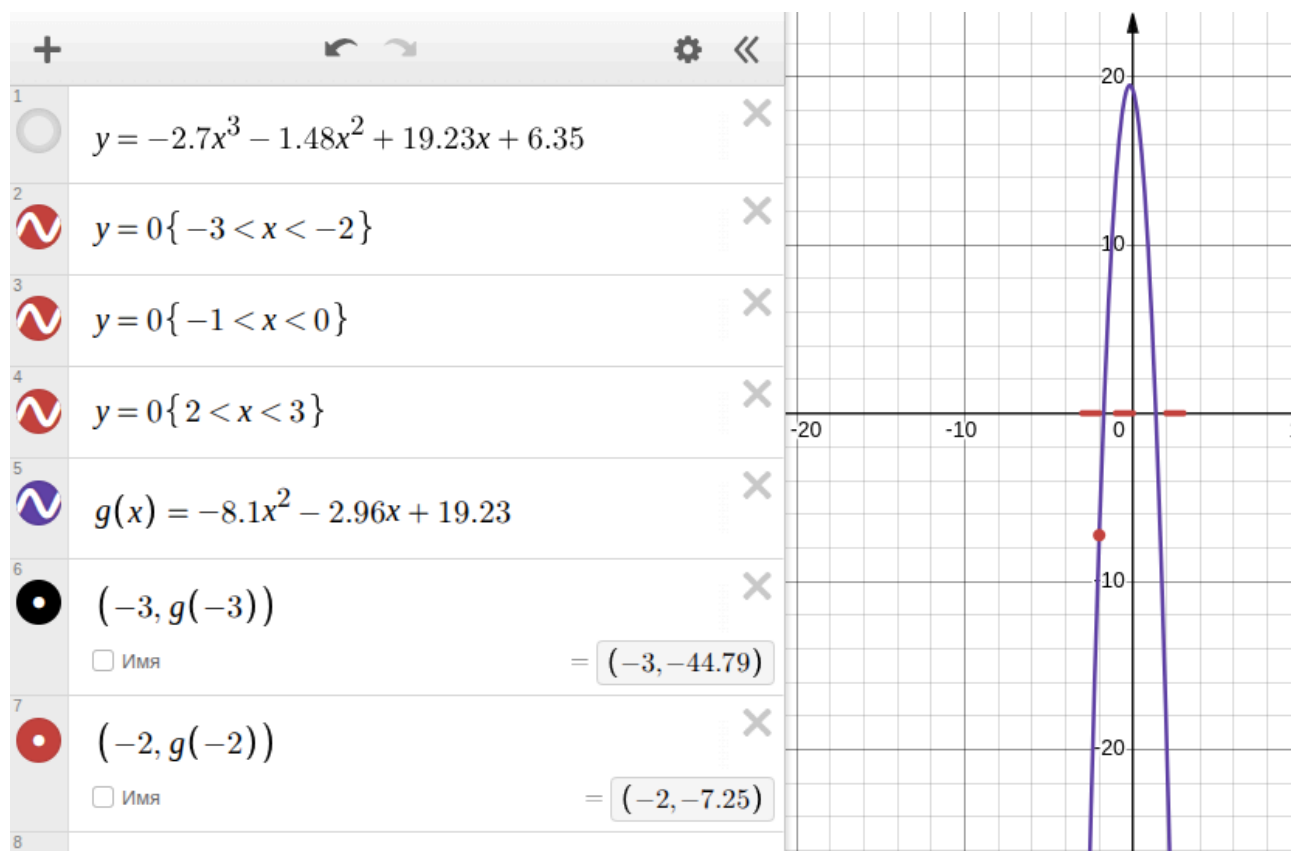


Рис. 3. График производной исходной функции

$$f'(-3) = -44,79$$

$$f'(-2) = -7,25$$

$$\lambda = \frac{1}{\max_{[a,b]} |f'(x)|} = \frac{1}{44,79} \approx 0,022$$

$$\begin{aligned} x &= x + \lambda f(x) = x + 0,022 \cdot (-2,7x^3 - 1,48x^2 + 19,23x + 6,35) \\ &= x - 0,0594x^3 - 0,03256x^2 + 0,42306x + 0,1397 \end{aligned}$$

Получили функцию:

$$\varphi(x) = -0,0594x^3 - 0,03256x^2 + 1,42306x + 0,1397$$

Проверка сходимости:

$$\begin{aligned} \varphi'(x) &= 3 \cdot -0,0594x^2 + 2 \cdot -0,03256x + 1,42306 \\ &= -0,1782x^2 - 0,06512x + 1,42306 \end{aligned}$$

$$\varphi'(a) = \varphi'(-3) \approx 0,015 < 1$$

$$\varphi'(b) = \varphi'(-2) \approx 0,841 < 1$$

Условие сходимости выполняется.

Пример вычисления первого шага итерации:

В качестве начального приближения выберем $x_0 = a = -3$, так как $f(a) \cdot f''(a) > 0$.

$$x_1 = \varphi(x_0) = -0,0594 \cdot (-3)^3 - 0,03256 \cdot (-3)^2 + 1,42306 \cdot (-3) + 0,1397$$

$$x_1 = 1,6038 - 0,29304 - 4,26918 + 0,1397 \approx -2,819$$

Оценим погрешность и значение функции:

$$|x_1 - x_0| = |-2,819 - (-3)| = 0,181$$

$$f(x_1) \approx 0,854$$

№ шага	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-3	-2,819	0,854	0,181
2	-2,819	-2,800	0,069	0,019
3	-2,800	-2,798	0,005	0,002

Таблица 2. Таблица вычислений для метода простой итерации

Ответ: $-2,798$ при $\varepsilon = 0,01$

2.1.3. Уточнение центрального корня методом секущих

$$a = -1, b = 0$$

$$f(x) = -2,7x^3 - 1,48x^2 + 19,23x + 6,35$$

Рабочая формула метода:

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \cdot f(x_k)$$

Выберем начальное приближение. В силу того, что:

$$f(a) \cdot f''(a) < 0 \text{ и } f(b) \cdot f''(b) < 0$$

Однозначной сходимости не гарантировано. Но все же попробуем выбрать x_0 и близкое к нему $x_1 = x_0 + \varepsilon$.

Пусть $x_0 = b = 0$, тогда $x_1 = -0,01$.

Вычисляем значения функции:

$$f(x_0) = f(0) = 6,35$$

$$f(x_1) = f(-0,01) \approx 6,158$$

Первая итерация ($k = 1$):

$$\frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{6,158 - 6,35}{-0,01 - 0} = \frac{-0,192}{-0,01} = 19,2$$

$$x_2 = -0,01 - \frac{6,158}{19,2} \approx -0,330$$

№ шага	x_{k-1}	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	0	-0,01	-0,330	-0,060	0,320
2	-0,01	-0,330	-0,327	0,000	0,003

Таблица 3. Таблица вычислений для метода секущих

Ответ: 0,327 при $\varepsilon = 0,01$

2.2. Решение системы нелинейных уравнений

Решаем систему методом Ньютона

$$\begin{cases} \operatorname{tg}(xy + 0.3) = x^2 \\ 0.9x^2 + 2y^2 = 1 \end{cases} \Leftrightarrow \begin{cases} \operatorname{tg}(xy + 0.3) - x^2 = 0 \\ 0.9x^2 + 2y^2 - 1 = 0 \end{cases} \Leftrightarrow \begin{cases} f(x, y) = 0 \\ g(x, y) = 0 \end{cases} \stackrel{\text{def}}{=}$$

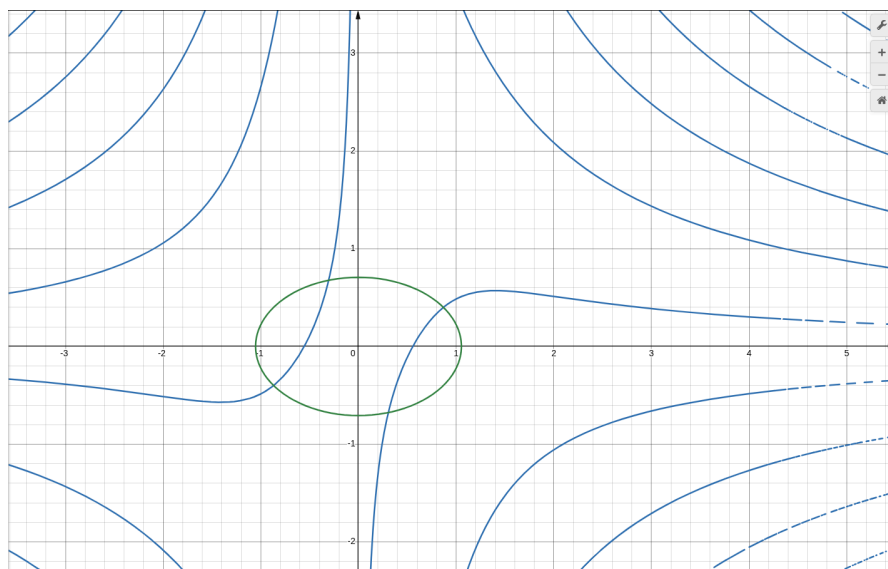


Рис. 4. График системы уравнений вар. 5

Находим частные производные и строим Якобиан:

$$\frac{\delta f}{\delta x} = \frac{y}{\cos^2(xy + 0.3)} - 2x; \quad \frac{\delta f}{\delta y} = \frac{x}{\cos^2(xy + 0.3)}$$

$$\frac{\delta g}{\delta x} = 1, 8x; \quad \frac{\delta g}{\delta y} = 4y$$

$$\begin{pmatrix} \frac{\delta f}{\delta x} & \frac{\delta f}{\delta y} \\ \frac{\delta g}{\delta x} & \frac{\delta g}{\delta y} \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} \frac{y}{\cos^2(xy+0.3)} - 2x & \frac{x}{\cos^2(xy+0.3)} \\ 1.8x & 4y \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} \stackrel{(*)}{=} \begin{pmatrix} f(x, y) \\ g(x, y) \end{pmatrix}$$

Получаем новую систему с приближениями $\Delta x, \Delta y$:

$$\begin{cases} \frac{y\Delta x}{\cos^2(xy+0.3)} - 2x\Delta x + \frac{x\Delta y}{\cos^2(xy+0.3)} = x^2 - \operatorname{tg}(xy + 0.3) \\ 1.8x\Delta x + 4y\Delta y = 1 - 0.9x^2 - 2y^2 \end{cases}$$

Вообще, из графика видно, что корней 4. Я приведу пример нахождения 1 корня. Остальные находятся аналогично, стоит лишь выбрать другое начальное приближение (визуально ближе к соответствующему корню)

Первая итерация. Пусть начальное приближение:

$$x_0 = -0.3, y_0 = 0.7$$

.

- Заранее вычислим значение аргумента тригонометрических функций:

$$x_0 y_0 + 0.3 = (-0.3) \cdot 0.7 + 0.3 = -0.21 + 0.3 = 0.09$$

$$\cos^2(0.09) \approx 0.9919$$

$$\operatorname{tg}(0.09) \approx 0.0902$$

- Вычисляем значения функций:

$$f(x_0, y_0) = 0.0902 - (-0.3)^2 = 0.0902 - 0.09 = 0.0002$$

$$g(x_0, y_0) = 0.9(-0.3)^2 + 2(0.7)^2 - 1 = 0.081 + 0.98 - 1 = 0.061$$

- Элементы матрицы Якоби:

$$\frac{\delta f}{\delta x}(x_0, y_0) = \frac{0.7}{0.9919} - 2(-0.3) = 0.7057 + 0.6 = 1.3057$$

$$\frac{\delta f}{\delta y}(x_0, y_0) = \frac{-0.3}{0.9919} = -0.3024$$

$$\frac{\delta g}{\delta x}(x_0, y_0) = 1.8 \cdot (-0.3) = -0.54$$

$$\frac{\delta g}{\delta y}(x_0, y_0) = 4 \cdot 0.7 = 2.8$$

- Теперь решаем обычную СЛАУ:

$$\begin{cases} 1.3057\Delta x - 0.3024\Delta y = -0.0002 \\ -0.54\Delta x + 2.8\Delta y = -0.061 \end{cases}$$

- Из второго уравнения: $2.8\Delta y = -0.061 + 0.54\Delta x$.
- Решаем систему с помощью первой лабы, получаем:

$$\Delta x \approx -0.0054$$

$$\Delta y \approx -0.0228$$

- Получаем новые приближения:

$$x_1 = x_0 + \Delta x = -0.3 + (-0.0055) = -0.3055$$

$$y_1 = y_0 + \Delta y = 0.7 + (-0.0228) = 0.6772$$

- Проверяем точность

$$\begin{cases} |x_1 - x_0| = |-0.3055 + 0.3| \approx 0.05 > \varepsilon = 0.01 \\ |y_1 - y_0| = |0.6772 - 0.7| \approx 0.02 > \varepsilon = 0.01 \end{cases} \Rightarrow \text{считаем дальше}$$

Вторая итерация. Текущее приближение:

$$x_1 = -0.3055, y_1 = 0.6772$$

.

- Значения функций:

$$f(x_1, y_1) \approx 0.0001$$

$$g(x_1, y_1) \approx 0.0012$$

- Элементы матрицы Якоби в точке (x_1, y_1) :

$$J = \begin{pmatrix} 1.2941 & -0.3082 \\ -0.5499 & 2.7088 \end{pmatrix}$$

- Составляем СЛАУ для поправок:

$$\begin{cases} 1.2941\Delta x - 0.3082\Delta y = -0.0001 \\ -0.5499\Delta x + 2.7088\Delta y = -0.0012 \end{cases}$$

- Решаем систему:

$$\Delta x \approx -0.0002$$

$$\Delta y \approx -0.0005$$

- Новые приближения:

$$x_2 = x_1 + \Delta x = -0.3055 - 0.0002 = -0.3057$$

$$y_2 = y_1 + \Delta y = 0.6772 - 0.0005 = 0.6767$$

- Проверяем точность:

$$\begin{cases} |x_2 - x_1| \approx 0.0002 < \varepsilon \\ |y_2 - y_1| \approx 0.0005 < \varepsilon \end{cases} \Rightarrow \text{точность достигнута!!}$$

Первый корень: $x \approx -0.306; y \approx 0.677$.

Заметим, что корни симметричны относительно начала координат.

Аналогично находятся и остальных 3 корня:
 $(-0.87; -0.4), (0.306; -0.677), (0.87; 0.4)$

Ответ: $(-0.306; 0.677), (-0.87; -0.4), (0.306; -0.677), (0.87; 0.4)$.

3. Программная реализация задачи

Программа представляет собой клиент-серверное приложение: backend-часть написана на Golang, frontend - Vue.js

Все вычисления и решения систем происходят на backend в отдельных методах, которые будут представлены ниже. Фронтенд знает об уравнениях на бэкенде, поэтому помимо отправки запросов полностью отвечает за рисование графиков.

Сначала была идея рисовать графики на бэкенде, но я отказался от этой идеи в пользу генерации curving point - точек, по которым фронтенд строит кривую. Однако впоследствии, чтобы излишне не усложнять логику и не добавлять ответственности бэкенду, все графики полностью рисуются на фронтенде.

Полный код: github.com/xtern00/vm_labs/tree/main/vm_lab2/backend

3.1. Функции решения нелинейных уравнений

3.1.1. Метод дихотомии

```
func (s *HalfDivisionSolver) Solve(  
    f equations.Equation,  
    a float64,  
    b float64,  
    eps float64,  
    maxIter int,  
) (Result, error) {  
    messages := make([]string, 0)  
  
    if a > b {  
        a, b = b, a  
        messages = append(messages, fmt.Sprintf("a and b were swapped (a <=  
b): a=%f, b=%f", a, b))  
    }  
  
    err := BasicCheck(f, a, b)  
    if err != nil {  
        return Result{}, err  
    }  
  
    steps := []Point{}  
  
    x := (a + b) / 2  
    steps = append(steps, Point{x, f(x)})
```

```

iter := 1
for {
    if math.Abs(a-b) <= eps || math.Abs(f(x)) < eps {
        break
    }
    if f(a)*f(x) > 0 {
        a = x
    } else {
        b = x
    }
    x = (a + b) / 2

    fx := f(x)
    if math.IsNaN(fx) || math.IsInf(fx, 0) {
        return Result{}, fmt.Errorf("func is not defined at x=%f", x)
    }
    steps = append(steps, Point{x, fx})

    iter++
    if iter > maxIter {
        return Result{}, errors.New("iter limit")
    }
}

return Result{
    Solution:    x,
    Value:       f(x),
    Iterations:  iter,
    ArgError:    math.Abs(a - b),
    Messages:    messages,
    Steps:       steps,
}, nil
}

```

3.1.2. Метод Ньютона

```

func (s *NewthonSolver) Solve(
    f equations.Equation,
    a float64,
    b float64,
    eps float64,
    maxIter int,
) (Result, error) {
    messages := make([]string, 0)

    if a > b {

```

```

    a, b = b, a
    messages = append(messages, fmt.Sprintf("a and b were swapped (a <=
b): a=%f, b=%f", a, b))
}

err := BasicCheck(f, a, b)
if err != nil {
    return Result{}, err
}

var xPrev float64
if f(a)*numeric.SecondDerivative(f, a) > 0 {
    xPrev = a
    messages = append(messages, "x_0 = a, так как f(a)f''(a)>0")
} else if f(b)*numeric.SecondDerivative(f, b) > 0 {
    xPrev = b
    messages = append(messages, "x_0=b, так как f(b)f''(b)>0")
} else {
    xPrev = a
    messages = append(messages, "быстрая сходимость не гарантирована,
за начальное приближение выбрана точка x_0=a")
}

xCurr, err := getNextX(f, xPrev)
if err != nil {
    return Result{}, err
}

steps := []Point{}
steps = append(steps, Point{xCurr, f(xCurr)})

iter := 0
for ; iter < maxIter && math.Abs(xCurr-xPrev) > eps &&
math.Abs(f(xCurr)) > eps; iter++ {
    xPrev = xCurr
    xCurr, err = getNextX(f, xCurr)
    if err != nil {
        return Result{}, err
    }
    steps = append(steps, Point{xCurr, f(xCurr)})
}

if iter >= maxIter && math.Abs(xCurr-xPrev) > eps &&
math.Abs(f(xCurr)) > eps {
    return Result{}, errors.New("iter limit")
}

```

```

}

return Result{
    Solution:  xCurr,
    Value:     f(xCurr),
    Iterations: iter,
    ArgError:  math.Abs(xCurr - xPrev),
    Messages:  messages,
    Steps:     steps,
}, nil
}

func getNextX(f equations.Equation, x float64) (float64, error) {
    deriv := numeric.Derivative(f, x)
    if math.Abs(deriv) <= minDerivative {
        return 0, fmt.Errorf("производная слишком близка к 0 в точке x=%f",
x)
    }

    return x - (f(x) / deriv), nil
}

```

3.1.3. Метод простых итераций

```

func (s *SimpleIterationsSolver) Solve(
    f equations.Equation,
    a float64,
    b float64,
    eps float64,
    maxIter int,
) (Result, error) {

    messages := make([]string, 0)

    if a > b {
        a, b = b, a
        messages = append(messages, fmt.Sprintf("a and b were swapped (a <=
b): a=%f, b=%f", a, b))
    }

    err := BasicCheck(f, a, b)
    if err != nil {
        return Result{}, err
    }

    derivativeFA := numeric.Derivative(f, a)
    derivativeFB := numeric.Derivative(f, b)

```

```

lambda := 1 / math.Max(derivativeFA, derivativeFB)
if derivativeFA > 0 && derivativeFB > 0 {
    lambda *= -1
}

phi := func(x float64) float64 {
    return x + lambda*f(x)
}
if numeric.Derivative(phi, a) >= 1 || numeric.Derivative(phi, b) >= 1
{
    messages = append(messages, fmt.Sprintf("достаточное условие
сходимости не выполнено - сходимость не гарантирована, q = %f > 1.
Попытаемся вычислить корень", math.Max(derivativeFA, derivativeFB)))
}

steps := []Point{}

xPrev := a
xCurr := phi(xPrev)

iter := 0
for ; iter < maxIter && math.Abs(xCurr-xPrev) > eps; iter++ {
    xPrev = xCurr
    xCurr = phi(xCurr)

    steps = append(steps, Point{xCurr, f(xCurr)})
}
if iter >= maxIter && math.Abs(xCurr-xPrev) > eps {
    return Result{}, errors.New("iter limit")
}
if math.IsNaN(xCurr) || math.IsInf(xPrev, 0) {
    return Result{}, fmt.Errorf("функция phi не определена на %f. Метод
не сходится", xPrev)
}
if math.IsNaN(f(xCurr)) {
    return Result{}, fmt.Errorf("функция не определена на %f. Метод не
сходится", xCurr)
}

return Result{
    Solution:  xCurr,
    Value:     f(xCurr),
    Iterations: iter,
    ArgError:  math.Abs(xCurr - xPrev),
    Messages:  messages,
}

```



```

    Steps:      steps,
  }, nil
}

```

3.2. Решение системы нелинейных уравнений методом простых итераций

```

func (s *SystemSimpleIterSolver) Solve(
    system equations.System2,
    phiSystem equations.System2,
    x0 float64,
    y0 float64,
    eps float64,
    maxIter int,
) (System2Result, error) {
    f1 := system[0]
    f2 := system[1]
    if f1 == nil || f2 == nil {
        return System2Result{}, errors.New("system equations should not be
nil")
    }

    phi1 := phiSystem[0]
    phi2 := phiSystem[1]

    if phi1 == nil || phi2 == nil {
        return System2Result{}, errors.New("phi functions should not be
nil")
    }

    messages := []string{}

    // проверка условной сходимости в начальной точке
    dPhi1Dx := numeric.Derivative(
        func(v float64) float64 {
            return phi1(v, y0)
        }, x0,
    )
    dPhi1Dy := numeric.Derivative(
        func(v float64) float64 {
            return phi1(x0, v)
        }, y0,
    )
    dPhi2Dx := numeric.Derivative(
        func(v float64) float64 {
            return phi2(v, y0)
        }, x0,
    )

```

```

)
dPhi2Dy := numeric.Derivative(
    func(v float64) float64 {
        return phi2(x0, v)
    }, y0,
)
q1 := math.Abs(dPhi1Dx) + math.Abs(dPhi1Dy)
q2 := math.Abs(dPhi2Dx) + math.Abs(dPhi2Dy)
q := math.Max(q1, q2)
if math.IsNaN(q) || math.IsInf(q, 0) {
    messages = append(messages, "не удалось оценить достаточное условие
сходимости (q нечисловое)")
} else if q >= 1 {
    messages = append(messages, fmt.Sprintf("достаточное условие
сходимости не выполнено: q=%.6f >= 1; пробуем решить!", q))
}

steps := []Point{}
errPoints := []Point{}

xPrev, yPrev := x0, y0
xCurr, yCurr := phi1(x0, y0), phi2(x0, y0)
if math.IsNaN(xCurr) || math.IsInf(xCurr, 0) || math.IsNaN(yCurr) ||
math.IsInf(yCurr, 0) {
    return System2Result{}, fmt.Errorf("итерации вышли за зону
определения функции phi на первом шаге: x=%f, y=%f", xCurr, yCurr)
}

steps = append(steps, Point{xCurr, yCurr})
errPoints = append(errPoints, Point{math.Abs(xCurr - xPrev),
math.Abs(yCurr - yPrev)})

iter := 1
for ; iter < maxIter && math.Max(math.Abs(xPrev-xCurr),
math.Abs(yPrev-yCurr)) > eps && !checkSystemAns(system, xCurr, yCurr,
eps); iter++ {
    xPrev, yPrev = xCurr, yCurr
    xCurr, yCurr = phi1(xPrev, yPrev), phi2(xPrev, yPrev)
    if math.IsNaN(xCurr) || math.IsInf(xCurr, 0) || math.IsNaN(yCurr)
|| math.IsInf(yCurr, 0) {
        return System2Result{}, fmt.Errorf("итерации вышли за зону
определения функции phi на шаге %d: xCurr=%f, yCurr=%f, xPrev=%f,
yPrev=%f", iter, xCurr, yCurr, xPrev, yPrev)
    }

    steps = append(steps, Point{xCurr, yCurr})

```

```

    errPoints = append(errPoints, Point{math.Abs(xCurr - xPrev),
math.Abs(yCurr - yPrev)})

}

if iter >= maxIter && math.Max(math.Abs(xPrev-xCurr), math.Abs(yPrev-
yCurr)) > eps {
    return System2Result{}, errors.New("iter limit")
}

return System2Result{
    Solution:    Point{xCurr, yCurr},
    Iterations: iter,
    Messages:    messages,
    Steps:       steps,
    Errors:      errPoints,
}, nil
}

func checkSystemAns(s equations.System2, x float64, y float64, eps
float64) bool {
    f1Val := s[0](x, y)
    f2Val := s[1](x, y)
    return math.Abs(f1Val) < eps && math.Abs(f2Val) < eps
}

```

3.3. Результаты работы программы

Численное решение нелинейных уравнений и систем

[!] нет смены знака на [1.000000, 2.000000]: f(a)=-9.696428, f(b)=-16.796892. Значения на границах должны быть разных знаков

точность (eps): max iter:

НЕЛИНЕЙНЫЕ УРАВНЕНИЯ

☒ Уравнение 1
 $x^2 + e^x - 10x \sin x - 5x = 0$

☐ Уравнение 2
 $\ln x + (\sin x)^3 = 0$

☐ Уравнение 3
 $\sqrt{x} - 0.07x^3 + 0.5x^2 - 5 = 0$

☐ Уравнение 4
 $x^3 + 4x^2 + 3x - 1 = 0$

☐ Уравнение 5
 $-0.4x^3 + 2x^2 - 2 = 0$

ИНТЕРВАЛ [A; B]

a: b:

СПОСОБ РЕШЕНИЯ

☒ Метод половинного деления

☐ Метод Ньютона

☐ Метод простой итерации

СИСТЕМЫ НЕЛИНЕЙНЫХ УРАВНЕНИЙ

☐ Система 1
$$\begin{cases} \cos y + x - 1.5 = 0 \\ \sin x + y - 4 = 0 \end{cases}$$

☐ Система 2
$$\begin{cases} x^2 + y^2 - 1 = 0 \\ x^2 - 3y = 0.5 \end{cases}$$

НАЧАЛЬНОЕ ПРИБЛИЖЕНИЕ

x_0: y_0:

СПОСОБ РЕШЕНИЯ

☒ Метод простых итераций

Рис. 5. Пример обработки неточности выбранного интервала

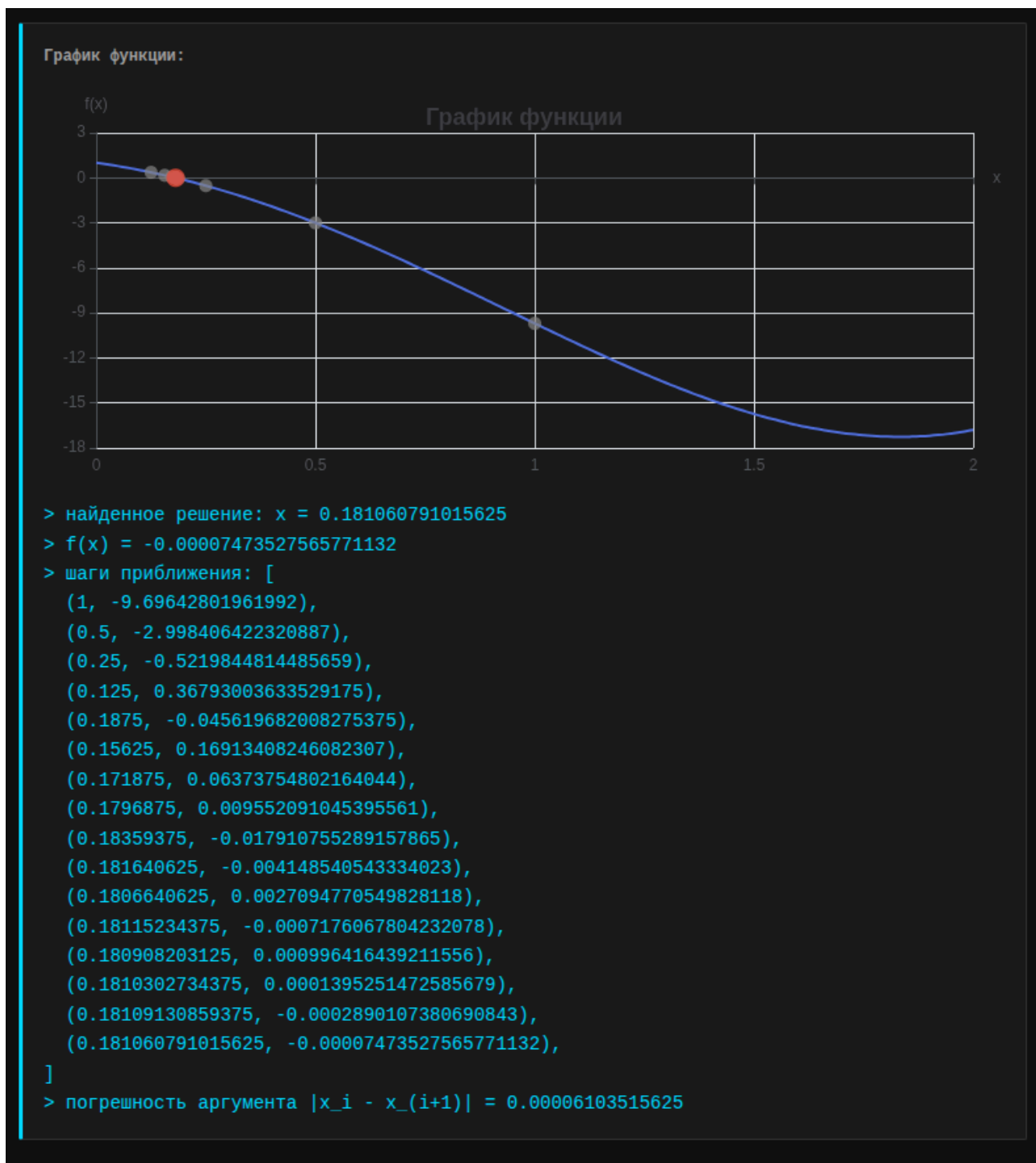
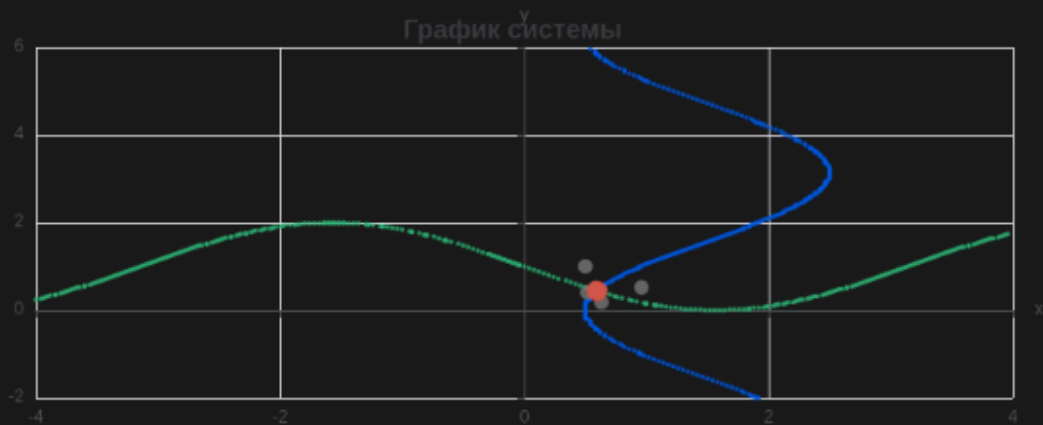


Рис. 6. Пример успешной работы программы нахождения корня нелинейного уравнения

График функции:



> найденное решение: (0.5950800891924847, 0.4394919822630634)

> количество итераций: 18

> шаги приближения: [

```
(0.5, 1),
(0.9596976941318602, 0.520574461395797),
(0.632466401956899, 0.18098184758612856),
(0.5163325611629134, 0.4088641153742405),
(0.5824269947235338, 0.5063058703267556),
(0.625458061390649, 0.4499475887761569),
(0.5995301018080966, 0.4145313175608659),
(0.5846048119809801, 0.43574541264159694),
(0.5934443342586788, 0.4480550823215972),
(0.5987086290861183, 0.4407802454723154),
(0.5955809494841152, 0.4364238115258109),
(0.5937308857056208, 0.439010227671642),
(0.594827196922406, 0.44054271170882353),
(0.5954796323445946, 0.4396343604784968),
(0.5950926569604572, 0.4390941031820439),
(0.5948628508685356, 0.4394145141847272),
(0.5949991103508768, 0.4396048306197915),
(0.5950800891924847, 0.4394919822630634),
```

]

> вектор погрешностей: [

```
(0.5, 1)
(0.45969769413186023, 0.479425538604203)
(0.3272312921749613, 0.33959261380966843)
(0.11613384079398559, 0.22788226778811194)
(0.06609443356062039, 0.09744175495251506)
(0.0430310666671152, 0.05635828155059863)
(0.02592795958255234, 0.035416271215291006)
(0.014835289827116571, 0.02121409508073102)
(0.008749522277698762, 0.012309669680000246)
(0.0052642948274395085, 0.007274836849281763)
(0.0031276796020031084, 0.004356433946504534)
(0.001850063778494393, 0.0025864161458311052)
(0.0010963112167852174, 0.0015324840371815363)
(0.0006524354221885131, 0.0009083512303267316)
(0.00038697538413734467, 0.0005402572964529062)
(0.00022980609192158052, 0.00032041100268331846)
(0.00013625948234119445, 0.00019031643506428608)
(0.0000809788416078483, 0.00011284835672809024)
```

]

Рис. 7. Пример успешной работы программы

4. Вывод

В ходе выполнения лабораторной работы я изучил численные методы решения нелинейных уравнений и систем нелинейных уравнений, используя Go для реализации алгоритмов. Из дополнительного, реализовал дискретную функцию поиска корней на промежутке. Также поработал с разными подходами к построению графиков: когда функция выражена в явном виде $y = f(x)$ – для нелинейных уравнений, и когда функция в неявном виде $f(x, y) = 0$ – для систем.